GPU shared memory

[M2 DiPaC] GPU Programming, Université Paris-Saclay – Session 5

Oguz Kaya

Assistant Professor
Université Paris-Saclay and LISN, Gif-sur-Yvette, France

université PARIS-SACLAY

Objectives

- Know the memory levels of a GPU: who sees each one, how fast it is
- Use **shared memory**: an on-chip array that the threads of a block share and manage themselves – and know when a **register** does the job better
- Use **thread synchronisation** properly, to avoid race conditions
- Identify **bank conflicts** – a coalescing-like problem, but in shared memory – and eliminate them through **padding**

Outline

1. Recap
2. Shared memory
3. A first example: `a[i]`^{k}
4. The 1D stencil and `__syncthreads()`
5. Bank conflicts
6. `a[i][j]`^{k} on a 2D array: coalescing, then banks
7. Exercises
8. Summary

Outline

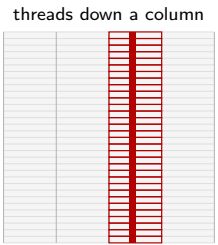
1. Recap
2. Shared memory
3. A first example: `a[i]`^{k}
4. The 1D stencil and `__syncthreads()`
5. Bank conflicts
6. `a[i][j]`^{k} on a 2D array: coalescing, then banks
7. Exercises
8. Summary

Recap: coalescing (session 3)

- A warp's load is **one request for 32 addresses**, served in **32-byte sectors** (8 floats)
- **Coalesced**: every fetched byte is used (e.g. 32 consecutive floats, `a[i]` ... `a[i + 31]`: 4 sectors, nothing wasted)
- A row-major matrix: threads **along a row** are coalesced; threads **down a column** fetch 32 sectors for 32 floats: **uncoalesced**



32 consecutive floats
4 sectors, all bytes used:
coalesced



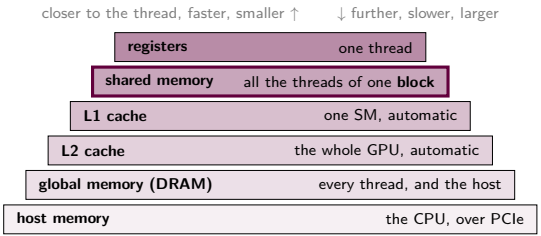
32 rows, $4n$ bytes apart
32 sectors, $\frac{1}{8}$ of each used:
not coalesced

a row-major array; grey lines: sector bounds (8 floats)

Outline

1. Recap
2. Shared memory
3. A first example: `a[i]`^{k}
4. The 1D stencil and `__syncthreads()`
5. Bank conflicts
6. `a[i][j]`^{k} on a 2D array: coalescing, then banks
7. Exercises
8. Summary

The memory levels a thread can use



- **Registers**: private to a thread, the fastest
- **Global memory**: seen by every thread and by the host, hundreds of cycles away; L1 and L2 cache it *automatically*
- **Shared memory**, in between: on the SM, seen by **all the threads of one block**, **managed by the program** – nothing is in it unless a thread puts it there. It eats into the L1 cache: the two share the same physical memory (L1 cache / SRAM)

How fast is each level?

level	seen by	size	latency	bandwidth
registers	one thread	256 KB register file	$\approx$ 0 cycles	$\approx$ 200 TB/s
shared memory	one block	up to 228 KB per SM	$\approx$ 30 cycles	$\approx$ 35 TB/s
L1 cache	one SM	the rest of a 256 KB pool	$\approx$ 35 cycles	$\approx$ 35 TB/s
L2 cache	all SMs	$\approx$ 126 MB	$\approx$ 250 cycles	$\approx$ 15 TB/s
DRAM (HBM3e)	all SMs, host	288 GB	$\approx$ 600 cycles	8 TB/s
host (PCIe 6)	–	4 TB of RAM	$\approx 10\,\mu$s	128 GB/s

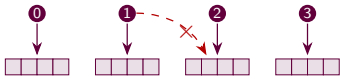
NVIDIA B300 ($\approx$ 150 SMs): memory sizes and the DRAM / PCIe bandwidths from NVIDIA's specifications; the on-chip bandwidths and the latencies are approximate (not published).

- Shared memory and L1 are the *same* memory, but a cache must first check its tags (is the line here? in which way?) and may miss; shared memory is addressed directly, never misses: a few cycles less, and **predictable**
- Multiple warps hide latency (session 1); **nothing hides bandwidth**. The only way: bring the data **closer to the registers**, and **reuse** it as much as possible

Shared memory: what it is

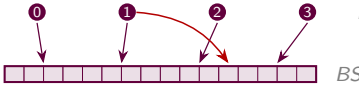
- A thread can declare a small array of its own, `float v[K]` (fixed `K`, loops unrolled): it lives in its **registers**, and **no other thread sees it**
- `__shared__` declares an array of `BS * K` floats for the **whole block**: one copy per block, **visible to all its threads**, each of which can read or modify **any element** when needed
- It lives as long as the block: no `cudaMalloc`, no copy, no `cudaFree`. **Not initialised**. In the same memory as L1: 48 KB per block by default (more by opting in, up to 227 KB of the SM's 228)

`float v[K]`: one array *per thread*, in its registers



BS private copies; thread 1 cannot read thread 2's cells

`__shared__ float s[BS * K]`: one array *per block*



in shared memory; every thread reads or writes *any* cell: thread 1 reads a cell thread 2 wrote

Declaring shared memory: static and dynamic

- **Static**: the size is a compile-time constant – a literal or a `#define`, never `blockDim.x`
- **Dynamic**: `extern`, and the size **in bytes** as the **third launch parameter**
- With `extern`, only **one** dynamic shared array per kernel (several arrays are carved from it with offsets)

```
1  // static: size known at compile time
2  __global__ void kernelS(float *a)
3  {
4    __shared__ float s[256];
5    ...
6  }
7
8  // dynamic: size given at launch
9  __global__ void kernelD(float *a)
10 {
11   extern __shared__ float s[];
12   ...
13 }
14
15 size_t bytes = 256 * sizeof(float);
16 kernelD<<<grid, block, bytes>>>(da);
```

Example: shared memory vs private registers

Left: each thread fills and reads **its own** `v[K]`. Right: each thread fills **its part** of `s`, then reads a cell **another thread** filled (the barrier between the two: the stencil section)

```
1 // K floats of my own: in registers, and
2 // invisible to the other threads
3 __global__ void privateArray(int n, float *a)
4 {
5   int i = blockIdx.x * blockDim.x + threadIdx.x;
6   float v[K];                    // mine only
7   if (i < n) {
8     for (int j = 0; j < K; j++) v[j] = a[i] + j;
9     a[i] = v[0] + v[K - 1];
10  }
11 }
```

```
1 // BS * K floats for the whole block: every
2 // thread can read and write any of them
3 __global__ void sharedArray(int n, float *a)
4 {
5   int t = threadIdx.x;
6   int i = blockIdx.x * blockDim.x + t;
7   __shared__ float s[BS * K];   // the block's
8   if (i < n) {
9     for (int j = 0; j < K; j++)
10      s[t * K + j] = a[i] + j;
11  }
12  __syncthreads();              // (see later)
13  int u = (t + 1) % BS;         // my neighbour
14  if (i < n) a[i] = s[t * K] + s[u * K + K - 1];
15 }
```

Outline

1. Recap
2. Shared memory
3. A first example: `a[i]`^{k}
4. The 1D stencil and `__syncthreads()`
5. Bank conflicts
6. `a[i][j]`^{k} on a 2D array: coalescing, then banks
7. Exercises
8. Summary

a[i] to the power k: a first kernel

- Replace every element a[i] by a[i]^k, with k multiplications. One thread per element
- As written, the loop reads a[i] from **global memory** k **times**: k loads, hundreds of cycles away, for **one** value
- Can each thread read it once and keep it close? Put it in **shared memory**

```
1  #define BS 256   // threads per block
2
3  // a[i] is read from global memory k times
4  __global__ void powerGlobal(int n, float *a, int k)
5  {
6    int i = blockIdx.x * blockDim.x + threadIdx.x;
7    if (i < n) {
8      float c = 1.0f;
9      for (int j = 0; j < k; j++) {
10       c *= a[i];          // global read
11     }
12     a[i] = c;             // global write
```

a[i]^k with shared memory

- s[BS]: one cell per thread of the block, a static size
- Each thread copies a[i] into s[t] **once**, then reads it k times **on chip**: ≈ 30 cycles each instead of ≈ 600

Is shared memory really needed here?

```
1  }
2
3  // a[i]: 1 global read, k shared reads
4  __global__ void powerShared(int n, float *a, int k)
5  {
6    __shared__ float s[BS];  // one per thread
7    int t = threadIdx.x;
8    int i = blockIdx.x * blockDim.x + t;
9    if (i < n) {
10     s[t] = a[i];           // global read
11     float c = 1.0f;
12     for (int j = 0; j < k; j++) {
13       c *= s[t];           // shared read
14     }
15     a[i] = c;              // global write
```

Shared memory is not needed here: a register

- Thread t reads only `s[t]`, the cell it wrote itself: **no thread needs another thread's data**
- Such data belongs in a **register**: faster still, and no shared memory taken from the SM
- **The rule**: shared memory pays when threads use data that *other* threads loaded – next example
- **In fact** `nvcc` sees that `a[i]` (first kernel) and `s[t]` (second) do not change inside the loop, and puts them in a register as here: the three kernels compile to the **same code**, and run in the same time

```
1 }
2
3 // a[i]: 1 global read, k register reads
4 __global__ void powerRegister(int n, float *a, int k)
5 {
6   int i = blockIdx.x * blockDim.x + threadIdx.x;
7   if (i < n) {
8     float x = a[i];        // global read
9     float c = 1.0f;
10    for (int j = 0; j < k; j++) {
11      c *= x;              // register read
12    }
13    a[i] = c;              // global write
```

Outline

1. Recap
2. Shared memory
3. A first example: `a[i]`^{k}
4. The 1D stencil and `__syncthreads()`
5. Bank conflicts
6. `a[i][j]`^{k} on a 2D array: coalescing, then banks
7. Exercises
8. Summary

The 1D stencil: a first kernel

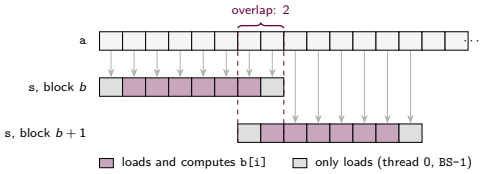
- `b[i] = (a[i-1] + a[i] + a[i+1]) / 3` for $0 < i < n - 1$: tp1's 3-point convolution. One thread per element
- **Three global reads** per output – and every element is read by **three threads**
- Threads **share data** here: can each element be read from global memory **once**, then shared on chip?

```
1 // 3 global reads per output
2 __global__ void stencilGlobal(
3     int n, const float *a, float *b)
4 {
5   int i = blockIdx.x * blockDim.x + threadIdx.x;
6   if (i > 0 && i < n - 1) {
7     b[i] = (a[i - 1] + a[i] + a[i + 1]) / 3.0f;
8   }
9 }
```

The 1D stencil with shared memory

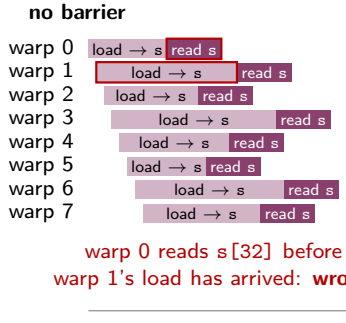
- Every thread **loads one element** into `s[t]`: the block loads BS contiguous elements
- Every thread computes `b[i]` from `s[t-1]`, `s[t]`, `s[t+1]` – except the two at the **extremities**, whose neighbours are not in `s`: the block computes **BS-2 elements**
- Hence block b computes the elements $b \cdot (\mathrm{BS}-2)+1 \ldots (b+1) \cdot (\mathrm{BS}-2)$, and the loads of consecutive blocks **overlap by 2**. (No `i < n` guard here: $n-2$ is a multiple of BS -2, so the blocks cover a exactly)

There is a problem with this kernel.
What could it be?



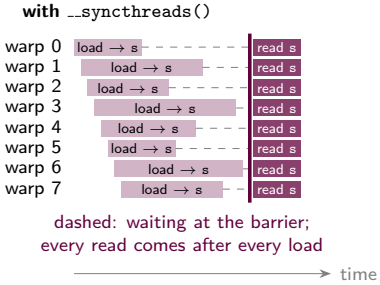

```
1 // Each thread loads one element, then computes
2 __global__ void stencilNoSync(
3     const float *a, float *b)
4 {
5   __shared__ float s[BS];
6   int t = threadIdx.x;
7   // consecutive blocks overlap by 2
8   int i = blockIdx.x * (BS - 2) + t;
9   s[t] = a[i];                    // 1 global read
10  if (t > 0 && t < BS - 1) {  // not the ends
11    b[i] = (s[t - 1] + s[t] + s[t + 1]) / 3.0f;
12  }
13 }
```

The problem: nobody waits for the other threads' loads



- `s[t] = a[i]` is a global load: the warp issues it and goes on; the value lands in `s` hundreds of cycles later. A warp waits only for **its own** loads
- Thread t reads `s[t-1]` and `s[t+1]`, loaded by other threads – at $t = 31, 32, 63, 64, \ldots$ by **another warp**, which runs in no fixed order
- Warp 0 may read `s[32]` before warp 1's load has arrived: a **race**. The result is wrong, and differently at each launch (`src/stencil-sync.cu`)

The fix: `__syncthreads()` just after the load

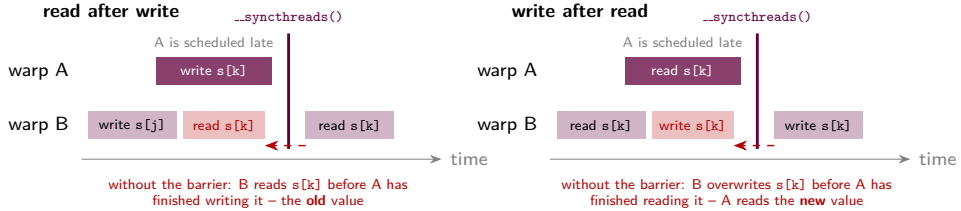



```
1 // The same, with a barrier after the load
2 __global__ void stencilShared(
3     const float *a, float *b)
4 {
5   __shared__ float s[BS];
6   int t = threadIdx.x;
7   // consecutive blocks overlap by 2
8   int i = blockIdx.x * (BS - 2) + t;
9   s[t] = a[i];                  // 1 global read
10  __syncthreads();              // s is complete
11  if (t > 0 && t < BS - 1) {  // not the ends
12    b[i] = (s[t - 1] + s[t] + s[t + 1]) / 3.0f;
13  }
14 }
```

- `__syncthreads()` is a **barrier for the block**: each thread waits there until **every thread of the block** has arrived. The writes made before it are then **visible** to all
- Placed just after the load: no thread reads `s` before every element is in it

Where was the barrier in tp1, or in `a[i]`^{k}? Nowhere: every thread read data written *before the kernel*, or by itself. The first time a thread reads what another thread of the same kernel wrote, a barrier appears.

Barriers: the rules



- **The pattern**: **load** → `__syncthreads()` → **use** → `__syncthreads()` before `s` is **overwritten** (next tile, next step)
- Needed between a write and *another thread's* read of the same word (**read after write**), and between a read and *another thread's* overwrite (**write after read**). Not for a thread's own words
- **Every thread of the block must reach every `__syncthreads()`.** One inside the stencil's `if (t > 0 && t < BS - 1)` is undefined: the block hangs, or computes garbage
- It synchronises **one block**. Blocks cannot wait for each other inside a kernel: the end of the kernel is the only barrier for the **grid** that we use

The 1D stencil: what did we gain?

kernel	global reads per output	ms	GB/s
`stencilGlobal`	3	0.281	237
`stencilShared`	≈ 1	0.303	220

`src/stencil-sync.cu`, $n \approx 8.3$ M floats, blocks of 256; Tesla T4 (cuda.godbolt.org), median of 5 runs. GB/s: `a` read once, `b` written once.

- One global read per element instead of three – but **little difference**: the **L1 cache** (the same memory as shared memory) already catches the repeated reads of a 3-point stencil, and both kernels wait for DRAM. The barrier and the 2 extra loads per block are not free
- Shared memory pays when the reuse is larger (a 2D stencil reads 5 or 9 neighbours, a 3D one 7 or 27), or when the reads would not be coalesced: next
- What this example teaches: **threads that share data need a barrier**

Outline

1. Recap
2. Shared memory
3. A first example: `a[i]`^{k}
4. The 1D stencil and `__syncthreads()`
5. Bank conflicts
6. `a[i][j]`^{k} on a 2D array: coalescing, then banks
7. Exercises
8. Summary

Shared memory is made of 32 banks

`__shared__ float a[128];` – 32 banks of 4 bytes; `a[w]` lives in **bank** w mod 32:
`a[0..31]` fill the 32 banks once, `a[32..63]` fill them again, ...

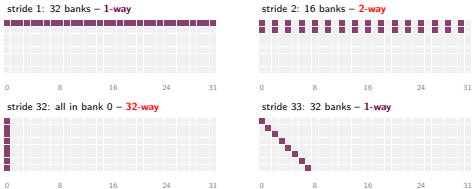
bank	0	1	2	3	4	5	...	26	27	28	29	30	31
a[0..31]	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	...	a[26]	a[27]	a[28]	a[29]	a[30]	a[31]
a[32..63]	a[32]	a[33]	a[34]	a[35]	a[36]	a[37]	...	a[58]	a[59]	a[60]	a[61]	a[62]	a[63]
a[64..95]	a[64]	a[65]	a[66]	a[67]	a[68]	a[69]	...	a[90]	a[91]	a[92]	a[93]	a[94]	a[95]
a[96..127]	a[96]	a[97]	a[98]	a[99]	a[100]	a[101]	...	a[122]	a[123]	a[124]	a[125]	a[126]	a[127]

`a[0]`, `a[32]`, `a[64]`, `a[96]`: four different words of bank 0 – one per pass

- **32 banks**, each 4 bytes wide: element w of a `float` (or `int`) array is in **bank** w mod 32. Each bank delivers **one word per pass**
- A warp access whose 32 words lie in 32 different banks takes one pass: **1-way**. k *distinct* words in one bank take k passes: a **k-way conflict**, $k\times$ the time
- For a 1-way access, the 32 threads **need not touch consecutive elements**: touching 32 different banks suffices
- Several threads reading the **same** word: read once, sent to all – a **broadcast**, no conflict

Strided access: 1-way, 2-way, . . . , 32-way


```
1 __shared__ float s[1024];
2 int t = threadIdx.x % 32; // 0 .. 31 in the warp
3 float x = s[stride * t]; // stride = 1, 2, 32, 33:
```



one line = 32 consecutive words, banks 0 . . . 31 from left to right (8 of the 32 lines shown: for stride 32, 8 of its 32 words); purple: the words read by the 32 threads of a warp, thread t reading word $t \cdot$ stride

stride	banks	conflict
1	32	1-way
2	16	2-way
4	8	4-way
8	4	8-way
16	2	16-way
32	1	32-way
3, 5, 7, . . .	32	1-way
33	32	1-way
0	1	broadcast

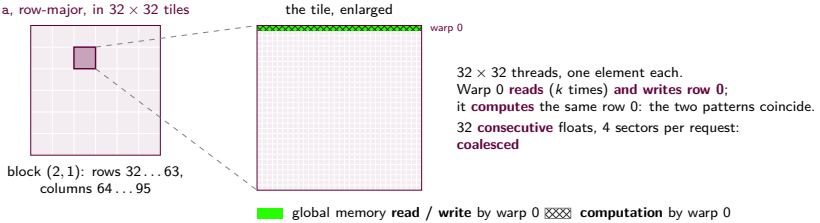
- Thread t reads `s[stride * t]`. **Even strides** fold the warp onto fewer banks; **odd strides** reach all 32
- Stride 32 is a **column** of a 2D array whose rows hold 32 floats: one bank, **32-way**

A k-way conflict costs k **passes**: $k\times$ the time of the access

Outline

1. Recap
2. Shared memory
3. A first example: `a[i]`^{k}
4. The 1D stencil and `__syncthreads()`
5. Bank conflicts
6. `a[i][j]`^{k} on a 2D array: coalescing, then banks
7. Exercises
8. Summary

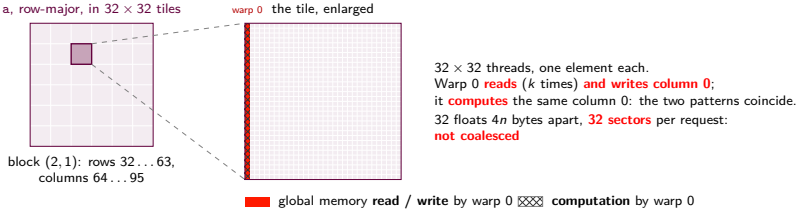
a[i][j]^k on a 2D array, horizontal warps: coalesced




```
1 // Horizontal warps: a warp handles a row
2 __global__ void powerH(float *a, int n, int k)
3 {
4   int tx = threadIdx.x, ty = threadIdx.y;
5   int i = blockIdx.y * T + ty;       // row
6   int j = blockIdx.x * T + tx;       // column
7   float c = 1.0f;
8   for (int p = 0; p < k; p++) { c *= a[i * n + j]; }
9   a[i * n + j] = c;
10 }
```

- a: $n \times n$ floats, row-major (a[i][j] is a[i * n + j]); every a[i][j] becomes a[i][j]^k. Blocks of 32×32 threads, one element per thread: each block owns a 32×32 **tile** of a
- **Horizontal warps**: thread (tx, ty) takes row ty, column tx of the tile. A warp (32 consecutive tx) touches a **row** of the tile: 32 consecutive floats, 4 sectors – **coalesced**, for the read and for the write

a[i][j]^k on a 2D array, vertical warps: not coalesced

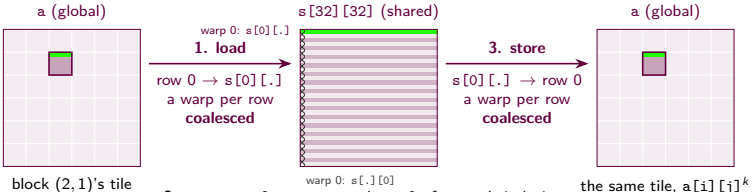



```
1 // Vertical warps: a warp handles a column
2 __global__ void powerV(float *a, int n, int k)
3 {
4   int tx = threadIdx.x, ty = threadIdx.y;
5   int i = blockIdx.y * T + tx;       // row
6   int j = blockIdx.x * T + ty;       // column
7   float c = 1.0f;
8   for (int p = 0; p < k; p++) { c *= a[i * n + j]; }
9   a[i * n + j] = c;
10 }
```

- **Vertical warps**: two lines change – thread (tx, ty) takes row tx, column ty. A warp touches a **column** of the tile: 32 floats 4n bytes apart, **32 sectors** per request – **not coalesced**. Same work: only the mapping changed
- Sometimes an array **has to be** walked by columns: a transposition (the rows of a are the columns of its transpose), tp3's kernel 5 (column-major B), . . .

How can we make the vertical kernel coalesced?

The vertical kernel through a shared tile




```
1  __global__ void powerTileV(float *a, int n, int k)
2  {
3    __shared__ float s[T][T];
4    int tx = threadIdx.x, ty = threadIdx.y;
5    int i = blockIdx.y * T + ty;
6    int j = blockIdx.x * T + tx;
7    s[ty][tx] = a[i * n + j];          // load: row
8    __syncthreads();
9    float c = 1.0f;                    // use: column
10   for (int p = 0; p < k; p++) { c *= s[tx][ty]; }
11   s[tx][ty] = c;
12   __syncthreads();
13   a[i * n + j] = s[ty][tx];          // store: row
14 }
```

- **Load horizontally**, `s[ty][tx]`: a warp per row, **coalesced**. Barrier
- **Use vertically**, `s[tx][ty]`: an element another warp loaded – no sectors in shared memory. Barrier
- **Store horizontally**, `s[ty][tx]`: a warp per row, **coalesced**
- Shared memory **dissociates** the direction of the loads and stores (rows: coalesced) from that of the computation (columns): **the best of both worlds**

The coalescing problem is gone. But **another problem** appears: which one?

A 32-way bank conflict

in each cell, the **bank** of the element: `s[r][c]` is word $32r + c$, in bank c *whatever the row*

`s[0][.]`	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
`s[1][.]`	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
`s[30][.]`	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
`s[31][.]`	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

warp 0 reads `s[0][0]`, `s[1][0]`, ..., `s[31][0]`: 32 different words, **all in bank 0** – 32 passes, a **32-way** conflict

- The load and the store, `s[ty][tx]`: a warp touches a **row** of `s`, 32 consecutive words in 32 banks: **1-way**
- The use, `s[tx][ty]`: a warp reads (and writes back) a **column** of `s`, 32 words **all in the same bank** (warp 0: bank 0): **32-way**, 32 passes – the coalescing problem has become a bank problem

Is there a way to fix this, keeping the vertical use?

Padding: `s[32][33]`

`s[r][c]` is now word $33r + c$, in bank $(r + c) \bmod 32$: each row starts **one bank further**; grey: the unused 33rd element

s[0][.]	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0
s[1][.]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	0	1
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
s[30][.]	30	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
s[31][.]	31	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31

warp 0 reads `s[0][0]`, `s[1][0]`, ..., `s[31][0]`: banks $0, 1, \ldots, 30, 31$ – **32 different banks**, one pass, **1-way**

One line changes (`powerTileVPadded`):

```
1 __shared__ float s[T][T + 1];
```

- **Pad** each row by one unused element: a column of `s` now falls in **32 different banks** (warp 0: banks $0, 1, \ldots, 31$) – the vertical use is **1-way**
- The horizontal load and store stay 1-way, and coalesced in global memory. The price: 32 unused floats per tile
- **All problems solved**: coalesced DRAM accesses, no bank conflict, the vertical mapping kept. **Rule**: a warp reading a column of a 2D shared array $\Rightarrow$ make the row length odd

Coalescing and bank conflicts: one question, two levels

	global memory (session 3)	**shared memory (today)**
served in units of	32-byte sectors	4-byte words, one per bank and pass
cost of a warp access	number of sectors fetched	largest number of distinct words in one bank
32 consecutive floats	4 sectors: **coalesced**	32 banks: **1-way**
the same address for all	1 sector	**broadcast**, 1 pass
stride 2, 4, . . . , 32 floats	8, 16, . . . , 32 sectors	**2-way**, **4-way**, . . . , **32-way**
odd stride (3, 5, . . . , 33)	12, 20, . . . , 32 sectors	**1-way**
a column of a 2D array	**32 sectors**	**32-way** if the row length is a multiple of 32
the remedy	change the mapping, or go through shared memory	**pad** the row to an odd length

- The same question at both levels: **how far apart are the addresses of consecutive threads?**
- A conflict costs passes inside the SM, not DRAM traffic – 32 of them for a column

Outline

1. Recap
2. Shared memory
3. A first example: `a[i]`^{k}
4. The 1D stencil and `__syncthreads()`
5. Bank conflicts
6. `a[i][j]`^{k} on a 2D array: coalescing, then banks
7. Exercises
8. Summary

Exercise: bank conflicts in a 1D array


```
1 __shared__ float s[1024];
2 int t = threadIdx.x % 32;
```

One warp, $t = 0 \dots 31$; `k` is the same for every thread. How many ways?

- (a) `s[t + 1]`
- (b) `s[2 * t]`
- (c) `s[3 * t]`
- (d) `s[31 - t]`
- (e) `s[t / 2]`
- (f) `s[32 * (t % 4)]`
- (g) `s[k]`

Exercise: bank conflicts in a 1D array


```
1 __shared__ float s[1024];
2 int t = threadIdx.x % 32;
```

One warp, $t = 0 \dots 31$; `k` is the same for every thread. How many ways?

- (a) `s[t + 1]`
- (b) `s[2 * t]`
- (c) `s[3 * t]`
- (d) `s[31 - t]`
- (e) `s[t / 2]`
- (f) `s[32 * (t % 4)]`
- (g) `s[k]`

- (a) **1-way**: 32 consecutive words (in global memory, the offset costs a sector)

Exercise: bank conflicts in a 1D array


```
1 __shared__ float s[1024];
2 int t = threadIdx.x % 32;
```

One warp, $t = 0 \dots 31$; `k` is the same for every thread. How many ways?

- (a) `s[t + 1]`
- (b) `s[2 * t]`
- (c) `s[3 * t]`
- (d) `s[31 - t]`
- (e) `s[t / 2]`
- (f) `s[32 * (t % 4)]`
- (g) `s[k]`

- (a) **1-way**: 32 consecutive words (in global memory, the offset costs a sector)
- (b) **2-way**: only the even banks, 2 words each

Exercise: bank conflicts in a 1D array


```
1 __shared__ float s[1024];
2 int t = threadIdx.x % 32;
```

One warp, $t = 0 \dots 31$; `k` is the same for every thread. How many ways?

- (a) `s[t + 1]`
- (b) `s[2 * t]`
- (c) `s[3 * t]`
- (d) `s[31 - t]`
- (e) `s[t / 2]`
- (f) `s[32 * (t % 4)]`
- (g) `s[k]`

- (a) **1-way**: 32 consecutive words (in global memory, the offset costs a sector)
- (b) **2-way**: only the even banks, 2 words each
- (c) **1-way**: banks $0, 3, 6, \dots, 30, 1, 4, \dots, 29$ – $3t$ mod 32 takes 32 different values; so does every odd stride

Exercise: bank conflicts in a 1D array


```
1 __shared__ float s[1024];
2 int t = threadIdx.x % 32;
```

One warp, $t = 0 \dots 31$; `k` is the same for every thread. How many ways?

- (a) `s[t + 1]`
- (b) `s[2 * t]`
- (c) `s[3 * t]`
- (d) `s[31 - t]`
- (e) `s[t / 2]`
- (f) `s[32 * (t % 4)]`
- (g) `s[k]`

- (a) **1-way**: 32 consecutive words (in global memory, the offset costs a sector)
- (b) **2-way**: only the even banks, 2 words each
- (c) **1-way**: banks $0, 3, 6, \dots, 30, 1, 4, \dots, 29$ – $3t$ mod 32 takes 32 different values; so does every odd stride
- (d) **1-way**: a permutation of 32 consecutive words

Exercise: bank conflicts in a 1D array


```
1 __shared__ float s[1024];
2 int t = threadIdx.x % 32;
```

One warp, $t = 0 \dots 31$; `k` is the same for every thread. How many ways?

- (a) `s[t + 1]`
- (b) `s[2 * t]`
- (c) `s[3 * t]`
- (d) `s[31 - t]`
- (e) `s[t / 2]`
- (f) `s[32 * (t % 4)]`
- (g) `s[k]`

- (a) **1-way**: 32 consecutive words (in global memory, the offset costs a sector)
- (b) **2-way**: only the even banks, 2 words each
- (c) **1-way**: banks $0, 3, 6, \dots, 30, 1, 4, \dots, 29$ – $3t$ mod 32 takes 32 different values; so does every odd stride
- (d) **1-way**: a permutation of 32 consecutive words
- (e) **1-way**: 16 words, each read by 2 threads – the same word is sent to both

Exercise: bank conflicts in a 1D array


```
1 __shared__ float s[1024];
2 int t = threadIdx.x % 32;
```

One warp, $t = 0 \dots 31$; `k` is the same for every thread. How many ways?

- (a) `s[t + 1]`
- (b) `s[2 * t]`
- (c) `s[3 * t]`
- (d) `s[31 - t]`
- (e) `s[t / 2]`
- (f) `s[32 * (t % 4)]`
- (g) `s[k]`

- (a) **1-way**: 32 consecutive words (in global memory, the offset costs a sector)
- (b) **2-way**: only the even banks, 2 words each
- (c) **1-way**: banks $0, 3, 6, \dots, 30, 1, 4, \dots, 29$ – $3t$ mod 32 takes 32 different values; so does every odd stride
- (d) **1-way**: a permutation of 32 consecutive words
- (e) **1-way**: 16 words, each read by 2 threads – the same word is sent to both
- (f) **4-way**: 4 distinct words 0, 32, 64, 96, all in bank 0: 4 passes; the 8 threads reading one word are served together (broadcast). Count *distinct words*, not threads

Exercise: bank conflicts in a 1D array


```
1 __shared__ float s[1024];
2 int t = threadIdx.x % 32;
```

One warp, $t = 0 \dots 31$; `k` is the same for every thread. How many ways?

- (a) `s[t + 1]`
- (b) `s[2 * t]`
- (c) `s[3 * t]`
- (d) `s[31 - t]`
- (e) `s[t / 2]`
- (f) `s[32 * (t % 4)]`
- (g) `s[k]`

- (a) **1-way**: 32 consecutive words (in global memory, the offset costs a sector)
- (b) **2-way**: only the even banks, 2 words each
- (c) **1-way**: banks $0, 3, 6, \dots, 30, 1, 4, \dots, 29$ – $3t$ mod 32 takes 32 different values; so does every odd stride
- (d) **1-way**: a permutation of 32 consecutive words
- (e) **1-way**: 16 words, each read by 2 threads – the same word is sent to both
- (f) **4-way**: 4 distinct words 0, 32, 64, 96, all in bank 0: 4 passes; the 8 threads reading one word are served together (broadcast). Count *distinct words*, not threads
- (g) **1-way**: one word for the whole warp, a broadcast

Exercise: bank conflicts in 2D arrays


```
1 __shared__ float s[32][32];
2 __shared__ float p[32][33];
3 __shared__ float q[32][48];
4 int t = threadIdx.x % 32;
```

Row-major: `s[r][c]` is word $32r + c$. One warp, `k` the same for all. How many ways?

- (a) `s[k][t]`
- (b) `s[t][k]`
- (c) `p[t][k]`
- (d) `s[t][t]`
- (e) `p[t][t]`
- (f) `q[t][k]`

Exercise: bank conflicts in 2D arrays


```
1 __shared__ float s[32][32];
2 __shared__ float p[32][33];
3 __shared__ float q[32][48];
4 int t = threadIdx.x % 32;
```

Row-major: `s[r][c]` is word $32r + c$. One warp, `k` the same for all. How many ways?

- (a) `s[k][t]`
- (b) `s[t][k]`
- (c) `p[t][k]`
- (d) `s[t][t]`
- (e) `p[t][t]`
- (f) `q[t][k]`

- (a) **1-way**: a row, 32 consecutive words

Exercise: bank conflicts in 2D arrays


```
1 __shared__ float s[32][32];
2 __shared__ float p[32][33];
3 __shared__ float q[32][48];
4 int t = threadIdx.x % 32;
```

Row-major: `s[r][c]` is word $32r + c$. One warp, `k` the same for all. How many ways?

- (a) `s[k][t]`
- (b) `s[t][k]`
- (c) `p[t][k]`
- (d) `s[t][t]`
- (e) `p[t][t]`
- (f) `q[t][k]`

- (a) **1-way**: a row, 32 consecutive words
- (b) **32-way**: a column, words $32t + k$, all in bank k

Exercise: bank conflicts in 2D arrays


```
1 __shared__ float s[32][32];
2 __shared__ float p[32][33];
3 __shared__ float q[32][48];
4 int t = threadIdx.x % 32;
```

Row-major: `s[r][c]` is word $32r + c$. One warp, `k` the same for all. How many ways?

- (a) `s[k][t]`
- (b) `s[t][k]`
- (c) `p[t][k]`
- (d) `s[t][t]`
- (e) `p[t][t]`
- (f) `q[t][k]`

- (a) **1-way**: a row, 32 consecutive words
- (b) **32-way**: a column, words $32t + k$, all in bank k
- (c) **1-way**: words $33t + k$, banks $(t + k) \bmod 32$ – the padding at work

Exercise: bank conflicts in 2D arrays


```
1 __shared__ float s[32][32];
2 __shared__ float p[32][33];
3 __shared__ float q[32][48];
4 int t = threadIdx.x % 32;
```

Row-major: `s[r][c]` is word $32r + c$. One warp, `k` the same for all. How many ways?

- (a) `s[k][t]`
- (b) `s[t][k]`
- (c) `p[t][k]`
- (d) `s[t][t]`
- (e) `p[t][t]`
- (f) `q[t][k]`

- (a) **1-way**: a row, 32 consecutive words
- (b) **32-way**: a column, words $32t + k$, all in bank k
- (c) **1-way**: words $33t + k$, banks $(t + k) \bmod 32$ – the padding at work
- (d) **1-way**: the diagonal, words $33t$, bank t

Exercise: bank conflicts in 2D arrays


```
1 __shared__ float s[32][32];
2 __shared__ float p[32][33];
3 __shared__ float q[32][48];
4 int t = threadIdx.x % 32;
```

Row-major: `s[r][c]` is word $32r + c$. One warp, `k` the same for all. How many ways?

- (a) `s[k][t]`
- (b) `s[t][k]`
- (c) `p[t][k]`
- (d) `s[t][t]`
- (e) `p[t][t]`
- (f) `q[t][k]`

- (a) **1-way**: a row, 32 consecutive words
- (b) **32-way**: a column, words $32t + k$, all in bank k
- (c) **1-way**: words $33t + k$, banks $(t + k)$ mod 32 – the padding at work
- (d) **1-way**: the diagonal, words $33t$, bank t
- (e) **2-way**: words $34t$, banks $2t$ mod 32. Padding repairs the columns, not every pattern

Exercise: bank conflicts in 2D arrays


```
1 __shared__ float s[32][32];
2 __shared__ float p[32][33];
3 __shared__ float q[32][48];
4 int t = threadIdx.x % 32;
```

Row-major: `s[r][c]` is word $32r + c$. One warp, `k` the same for all. How many ways?

- (a) `s[k][t]`
- (b) `s[t][k]`
- (c) `p[t][k]`
- (d) `s[t][t]`
- (e) `p[t][t]`
- (f) `q[t][k]`

- (a) **1-way**: a row, 32 consecutive words
- (b) **32-way**: a column, words $32t + k$, all in bank k
- (c) **1-way**: words $33t + k$, banks $(t + k)$ mod 32 – the padding at work
- (d) **1-way**: the diagonal, words $33t$, bank t
- (e) **2-way**: words $34t$, banks $2t$ mod 32. Padding repairs the columns, not every pattern
- (f) **16-way**: words $48t + k$ fall in only 2 banks, k and $k + 16$: 16 words each. Pad to an **odd** length: `q[32][49]`

Outline

1. Recap
2. Shared memory
3. A first example: `a[i]`^{k}
4. The 1D stencil and `__syncthreads()`
5. Bank conflicts
6. `a[i][j]`^{k} on a 2D array: coalescing, then banks
7. Exercises
8. Summary

What to remember

- **Shared memory**: one array per block, on chip ($\approx$ 30 cycles, $\approx$ 35 TB/s on the B300; DRAM: $\approx$ 600 cycles, 8 TB/s), **managed by the program**, not initialised. A static size, or `extern` and the size in bytes at launch
- It pays when threads use data that **other threads** loaded: **reuse** (stencils), **reorganise** (a tile loaded by rows, used by columns). Data one thread alone uses: a **register**
- `__syncthreads()`: load $\rightarrow$ barrier $\rightarrow$ use $\rightarrow$ barrier before overwriting. **Every thread of the block reaches every barrier**; blocks never wait for each other inside a kernel
- **Banks**: 32 banks of 4 bytes, word w in bank w mod 32; the cost is the largest number of distinct words in one bank. Consecutive words, broadcast, odd strides: 1-way. Stride 2: 2-way, ..., 32: 32-way. A column of a `[..][32]` array: 32-way $\Rightarrow$ **pad to `[..][33]`**
- Coalescing and bank conflicts ask the same question: how far apart are the addresses of consecutive threads?

This week's lab: tp4

Two programs, six kernels to write, skeletons provided; run on godbolt (Tesla T4) or on a lab GPU. A kernel is done when it prints `ok`.

1. **Matrix multiplication through shared memory** (`matmul-shmem.cu`, tp3's kernels given): each block walks along its bands of A and B, 32×32 tiles at a time, through padded shared tiles, both loads coalesced. Compare its GFLOP/s with kernel 4 and cuBLAS
2. **Matrix transposition** (`transpose.cu`): blocks of one thread; warps horizontal, then vertical on A; through a padded shared tile, A and A^T both touched horizontally; in place. Compare their GB/s with a copy (given)

- Today's tools: load $\rightarrow$ barrier $\rightarrow$ use $\rightarrow$ barrier, the bank rule, padding

Contact

Oguz Kaya
Université Paris-Saclay and LISN, Gif-sur-Yvette, France
oguz.kaya@universite-paris-saclay.fr
www.oguzkaya.com

Course material: `github.com/kayaogz/2627-m2dipac-gpuprogramming`